Index

Name : Mukesh Raja V Std. : Sec. :

Roll No. : Sub : CS23532 Computer Networks

S.No.	Date	Title	Page No.	Teacher's Sign / Remarks
1	17/7/26	Basic networking commands in Linux and Windows operating system		
2	17/7/26	~~Basi~~Learning and Assignment of IP addrs manually to computers.		
3	23/7/26	Study of different types of network cable and crimping of cable with RJ45 connector		
4	23/7/26	Implement packet sniffing using raw socket in python		
5	24/7/26	Develop a customised ping command to test the server connectivity		
9	13/8/26	Implement of subnetting in Cisco Packet Tracer		
10	13/8/26	Customize switch with network module using cisco packet tracer		
7	14/8/26	Develop a simple calculator using XML RPC		

exp no: 1 Basic Networking Commands in Linux and Windows operating system

17/7/2026

Aim:

To study and perform basic networking commands in Linux and windows operating system to understand network configuration, connectivity and troubleshooting

Linux Networking Commands

S.No	Command	Purpose	Output
1	ip addr	show IP address	2: eth0: inet 192.168.1.10/24
2	ip route	show routing table	default via 192.168.1.1 dev eth0
3	ifconfig	Display network interface	eth0 Link encap: Ethernet
4	ping google.com	Test connectivity	64 byte from 142.250.xx.xx: time = 15 ms
5	traceroute google.com	Trace packet route	192.168.1.1
6	nslookup google.com	DNS lookup	addres 142.250.xx.xx
7	dig google.com	Detail DNS info	Answer section: google.com IN A 142.250.xx.xx
8	host google.com	resolve hostname	google.com has adan 142.250.xx.xx
9	netstat -tuln	List listening port	tcp 0 0.0.0.0:22
10	ss -tuln	socket statistics	Listen 0 128 *:80
11	arp -a	view ARP cache	192.168.1.1 ether aa:bb:cc:dd:ee
12	hostname	show hostname	ubuntu-server
13	hostname -I	show IP address	192.168.1.10
14	curl ifconfig.me	show public IP	49.xxx.xxx.xxx

SNO	command	Purpose	output
15	wget https://exampl.com	Download webpage	saving to: index.html
16	tcpdump -i eth0	capture packets	IP 192.168.1.10 > 8.8.8.8
17	nmap 192.168.1.1	Scan port	22/tcp open ssh
18	ethtool eth0	NIC informa	speed: 1000Mb/s
19	nmcli dive stats	Network stats	eth0 connected
20	mtr google.com	Network diagnostics	loss % Avg Wrst

Windows Networkg commands

S.NO	Comnand	Purpose	output
1	ipconfig	show IP configuration	IPv4 Addr: 192.168.1.100
2	ipconfg/all	Detailed network info	physical Addres AA-BB-CC-DD-EE-FF
3	ipconfg/relase	Relase IP	succesfully released IP address
4	ipconfg/renew	Renew IP	succesfully renewed IP addres
5	tracert google.com	Trace route	1 <1ms 192.168.1.1
6	pathping google.com	Route & packet. loss	computing statistics
7	ipconfg/flushdns	clear DNS cash	succesfully flushed DNS resolver cash
8	nslookup google.com	DNS lookup	
10	netstat -an	show connection	TCP 0.0.0.0:80 LISTENING
11	arp -a	show ARP table	192.168.1.1 aa-bb-cc-dd-ee
12	route print	routing table	network Destination 0.0.0.0
13	hostname	show computer name	Desktop ABC123
14	getmac	Display MAC addres	AA-BB-CC-DD-EE-FF
15	net view	view shared compute	PC2

Sno	nb command	Purpose	output
16	nbtstat -n	Show Net BIOS statistics	NetBIOS Local Name table
17	net view	List computer on the network	//COMPUTER
18	net use	Manage network drive	the command completed succesfully
19	telnet google.com 80	Test TCP port connectivity	connected to google.com
20	HP. ftp.gnu.org	connect to an FTP server	connect to ftg.gnu.org

Result:

The basic networking commands in Linux and window were executed succesfully and their output were verified

exp no: 2 Learning and Assignment of
17/7/26 IP address manually to computer

Aim:

To manually assign an IP address, subnet mask, default gateway and DNS sever to a window computer and verify the network configuration

Procedure:

1. Open ~~network~~ control panel
2. Go to Network sharing center and select change adapter settings
3. Right click the active network adapter and select properties
4. select Internet protocol Version 4 (TCP/IPv4) and click properties
5. Use the following IP addres

IP address : 192.168.1.10
subnet mask : 255.255.255.0
Default Gateway : 192.168.1.1

7. Enter preferd DNS server
8. Click Ok to save the setting

~~Out~~

Command

ipconfig

output:

Window IP configuration

Enternet adapter Ethernet:

IPv4 Addm : 192.168.1.10

Subnet mant : 255.255.255.0

Dfaut Gateway : 192.168.1.1

9. open command prompt and type ipconfig to verify the assigned IP addm

10. close all

Result:

The computer was sucessfully configured with manual IP address and network connectivity was verified

exp:3 Study of different types of Network cables and Crimping of cable with RJ45 connector

23/7/26

Aim:

To study of different types of Network cable and crimping of cable with RJ45 connector

Different types of ~~cable~~ Network cable

1) UTP (Unshielded Twisted Pair)

consist of four twisted pairs of copper wires without shielding. common categories include cat5e cat6 and cat6a ~~1-40~~ Gbps / 100m distance (30m for cat 8)

Advantage:

* Low cost
* Light weight
* easy to install

Disadvantage

* More susceptible to electromagnetic interface and cross talk

Application:

LAN, homes, schools, office

Choose the Right Cable for Reliable & High-Performance Networks

1 UNSHIELDED TWISTED PAIR (UTP) CABLES

Most common cable for LAN, Ethernet, and office networks.

STRUCTURE

Outer Jacket

Twisted Pairs

KEY BENEFITS

- Commonly used in homes and offices
- Flexible, affordable, and easy to use
- For Ethernet and VoIP applications

2) STP (Shielded Twisted pair)

similar to UTP but includes shielding around the wires to reduce enterface 1Gbps - 10Gbps / 100m

advantage:

* Better protection
* higher signal quality

disadvantage:

* More expensive
* thicker
* Less flexible

application:

* factories
* data center
* hospital

3) Coaxial cable:

consist of a center copper conductor, insulating layer, metallic shield and outer jacket Maximum transmission speed is 10Mbps ~~to~~ upto 10Gbps / upto 500m

advantage:

* Good noise immunity
* support long distance than twisted pair durable.

Disadvantage:

* Bulky
* Difficult to instal
* lower bandwidth

2 SHIELDED TWISTED PAIR (STP) CABLES

Ideal for environments with high EMI and interference.

STRUCTURE

- Outer Jacket
- Foil Shielding
- Twisted Pairs

KEY BENEFITS

- Protect against interference and crosstalk
- Require proper grounding
- Ideal for environments with heavy EMI such as factories and industrial setups

3 COAXIAL CABLES

Used for traditional broadband, CCTV, and RF transmissions.

STRUCTURE

- Outer Protective Jacket
- Metallic Shield
- Copper Conductor

KEY BENEFITS

- Highly resistant to signal obstruction
- Great for long-distance transmissions
- Commonly used in data centers, ISP networks, and radio/broadcast applications

Application:

* cable TV
* cc TV
* broadband internet

Fibre optical cable:

Uses light singhals through glass or plastic fibre to transmit data at very high speeds 10 Gbps - several Tbps & upto 100km

Advantage:

* Very high bandwidth
* long distance communication
* secure transmission

Disadvantage:

* High installation cost
* Fragile
* require specifed tool

Application:

* Internet backbone
* telecommunication
* Data centers
* high speed networks

Result:

The study of different network cable was completed sucessfully and the RJ45 cable was crimped correctly with proper pin connectivity

4 FIBER OPTIC CABLES

Built for speed and performance in modern networks.

STRUCTURE

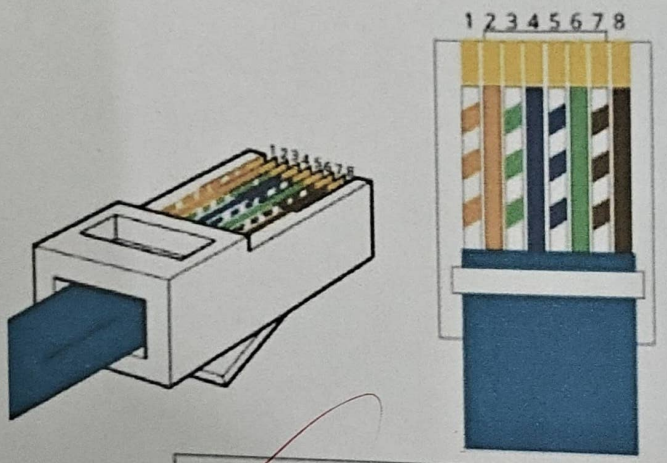
- Outer Jacket
- Glass Fiber Strands
- Light Signal Transmission

KEY BENEFITS

- Immune to electromagnetic interference
- Ideal for ultra-fast, long-range data transmission
- Widely used in data centers, ISP networks, and high-bandwidth applications

Tip: Selecting the right cable improves network performance, reduces downtime, and ensures a reliable network infrastructure.

RJ 45 Connector



1. White Orange	5. White Blue
2. Orange	6. Green
3. White Green	7. White Brown
4. Blue	8. Brown

1. White / orange
2. Orange
2. White / Green
4. Blue
5. White / Blue
6. Green
7. White / Brown
8. Brown

Crimping of cable with RJ45 connector

Material required:

- UTP cat 5e / cat 6 cable
- RJ 45 connector
- crimping tool
- wire stripper
- LAN cable Tester

Procedure:

1. CUT required length of UTP cable
2. strip about 2-3 cm of the outer jacket
3. Untwist and straighten the eight wire
4. Arrange the wire accordg to the T568B color sequence
5. Trim the wire evenly
6. Insert the wire completly Push the RJ45 connector in the crimping tool and press firmly
7. Place the connectr in the crimping tool and press firmly
8. Repeat the proces for the other end
9. Test the cable usig a LAN cable

Result:

The different types of Network cable were studied and the UTP cable was succesfully crimpled with the a RJ45 connector and tested sucessfully

EXP NO: 4 Implement Packet Sniffing Using Raw socket in Python

Date: 23/7/26

Aim:

To implement packet sniffing raw socket in python and observe the network packets transmitted and received by the system

Procedure:

1. Open a terminal or command prompt with administrator / root privileges
2. create a python program that use the socket module to create a raw socket.
3. Configure the raw socket to capture IP packets
4. continuously receive packets from the network interface
5. Extract and display basic packet information such as source IP, destination IP, protocol and packet length.

6. Run the program while generating network traffic

7. observe the captured packet details displayed on the screen

Code:

```
import socket
import struct
import os

def get_PN (proto_num):
    if proto_num == 1:
        return "ICMP"
    elif proto_num == 6:
        return "TCP"
    elif proto_num == 17:
        return "UDP"
    else:
        return "other"
```

Result:

The packet sniffer was succesfully implemented using python raw socket and network packet were captured and observed.

Sample output

Packet Sniffer started ...

Packet Received

Soure IP : 192.168.1.10
Destination IP : 142.250.183.78
Protocol : TCP
Packet : 60 bytes

Packet Recived

Soure IP : 8.8.8.8
Destination IP : 192.168.1.10
Protocol : ICMP
Packet Length : 84 byts

Packet Recived

Soure IP : 192.168.1.10
Destmeha IP : 172.217.167.110
Protocol : UDP
Packet leyth : 72 hytes

```
def main():
    proto = socket.IPPROTO_ if os.name == "nt" else
            socket.IPPROTO_ICMP
    sock = socket.socket(socket.AF_INET, socket.SOCK_RAW,
                         proto)
    sock.bind(('0.0.0.0', 0))
    sock.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
    print("Packet Sniffer started ... \n")
    try:
        while True:
            packet, addr = sock.recvfrom(65535)
            ip_header = packet[0:20]
            iph = struct.unpack('!BBHHHBBH4s4s', iph_hdr)

            t_l = iph[2]
            protocol_num = iph[6]
            src_ip = socket.inet_ntoa(iph[8])
            dst_ip = socket.inet_ntoa(iph[9])

            protocol_name = get_protocol_name(protocol_num)
            print("Packet Received")
            print('-' * 30)
            print("Source IP", src_ip)
            print("Destination IP", dst_ip)
            print("Protocol", protocol_name)
            print("Packet length", total_length, "\n")
    except KeyboardInterrupt:
        print("\n Packet sniffer stopped.")
    finally:
        sock.close
```

Result:

The packet sniffer was successfully implemented using python raw sockets, and network packets were captured and observed.

expno:5 Develop a customized Ping Command
24/7/26 to test server connectivity

Aim:

To Develop a customized Ping Command to test server connectivity

Theory:

Introduction:

The ping command is a basic network diagnostic tool used to verify wheather a server or another network device is reachable

Why we are using Ping command

We use the ping command to

* check wheather a server (or) host is reachable over network
* verify that the network or local network connection is working
* Identify network connection problem
* Measure time taken for data to travel to the destination and back

ex: Ping google.com

Post

Purpose of the Ping command

The main purpose of ping command is to test server connectivity

objective:

- Verifying whether a host is online
- Test communication between two device
- Measure network latency
- Detecting packet loss

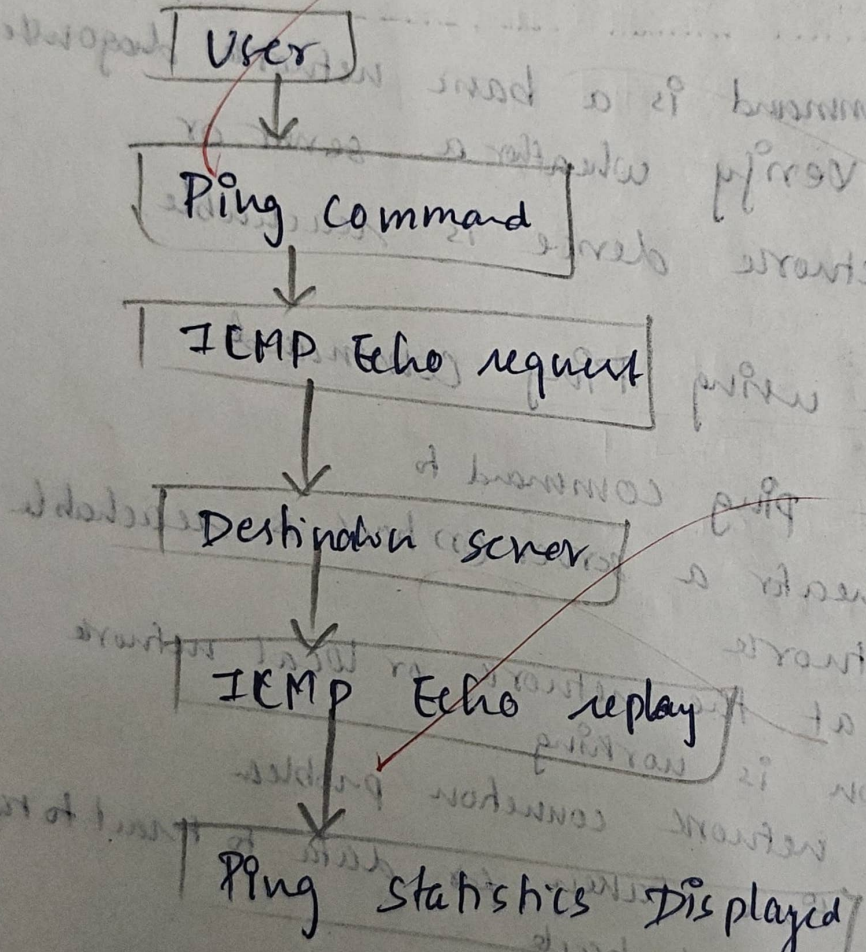
How does Ping establish communication

The ping command work using the Internet control Message Protocol (ICMP)

steps: ~~The~~

1. User enter the ping command
   ping example.com
2. The domain name is converted into an IP address using DNS
3. The ping utility send an ICMP Echo request packet to the destination server
4. If the destination server is reachable and allow ICMP traffic, it send back an ICMP Echo replay packet
5. The ping utility meeure:
   - Time taken for reply
   - Number of packet sent
   - Number of packet recived
   - Packet loss percentage
6. The results are displayed to the user

Communication flow



Ping commands:

Purpose	Windows command	Linux command	Description
Basic Ping	ping google.com	ping google.com	send ICMP Echo request packet to check whether target host is reachable
specific Number of packet	ping google.com -n 5	ping -c 5 google.com	send exactly 5 Ping request to the destination and then stop automatically
continous Ping	ping google.com -t	Ping google.com	continously send ping request until interrupted (ctrl+c)
set packet size	ping google.com -l 1024	ping -s 1024 google.com	send packets with a paload size of 1024 byte to test packet transmission
set time out	ping google.com -w 3000	ping -w 3 google.com	set the maximum time to wait for a reply windows-ms, Linux (s)
resolve hostname to IP	ping google.com	ping google.com	Automatically resolve the domain name to its IP addr
Ping using IP addr	ping 8.8.8.8	ping 8.8.8.8	Test connecting directly to an IP addr bypass DNS name resolution
set time-to-live (TTL)	Ping google.com -i 64	ping -t 64 google.com	set the TTL value of the ICMP packet

Command:

ping google.com

Sample output

Pinging google.com [142.250.67.46] with 32 byte of data:

Reply from 142.250.67.46: bytes=32 time=13ms TTL=118

Reply from 142.250.67.46 byte=32 time=18ms TTL=118

Reply from 142.250.67.46 byte=32 time=12ms TTL=118

Reply from 142.250.67.46 byte=32 time=10ms TTL=118

Ping statistics for 142.250.67.46

Packet: sent=4, Recived=4 Lost=0 (0% loss)

Approximate round trip time in milli seconds

Minimum=10ms, Maximum=18ms, Average=13ms

Result

Thus the customized Ping command to test server connectivity is sucessfully developed and verified.

Ex.No. 9 Implementation of subnetting in
Date: 13/8/26 Cisco packet tracer

Aim

To implementation of SUBNETTING in CISCO Packet tracer simulator

Theory:

classless IP subnetting divides an IP network into smaller subnets using flexible subnet mask instead of default class-based mask. This improve IP address utilization

creating Network Topology:

Create a new network in Cisco packet Tracer and add the required routers, switches and PC's. Connect the devices using suitable cables

subnetting:

For the network 192.168.1.0/24 a /27 subnet mask (255.255.255.224) can be used. It provides 8 subnets with 30 usable host address per subnet, sufficient for the required devices.

The IP addressing for the network show in the topology follow: Router R1:

Gigabit Ethernet 0/0 : 192.168.1.1
Gigabit Ethernet 0/1 : 192.168.2.1

switch S1:

Fast Ethernet 0/1 : 192.168.1.0/27
PC1 : 192.168.1.11
PC2 : 192.168.1.12
PC3 : 192.168.1.13
PC4 : 192.168.1.14
PC5 : 192.168.1.15

FastEthernet 0/2 : 192.168.2.0/27
PC1 : 192.168.2.11
PC2 : 192.168.2.12
PC3 : 192.168.2.13
PC4 : 192.168.2.14
PC5 : 192.168.2.15

Router R2 :

FasterEthernet 0/0 : 192.168.3.1
FasterEthernet 0/1 : 192.168.4.1

switch S2 :

FastEthernet 0/1 : 192.168.3.0/27
PC1 : 192.168.3.11
PC2 : 192.168.3.12
PC3 : 192.168.3.13
PC4 : 192.168.3.14
PC5 : 192.168.3.15

FastEthernet 0/2 : 192.168.4.0/27
PC1 : 192.168.4.11
PC2 : 192.168.4.12
PC3 : 192.168.4.13
PC4 : 192.168.4.14
PC5 : 192.168.4.15

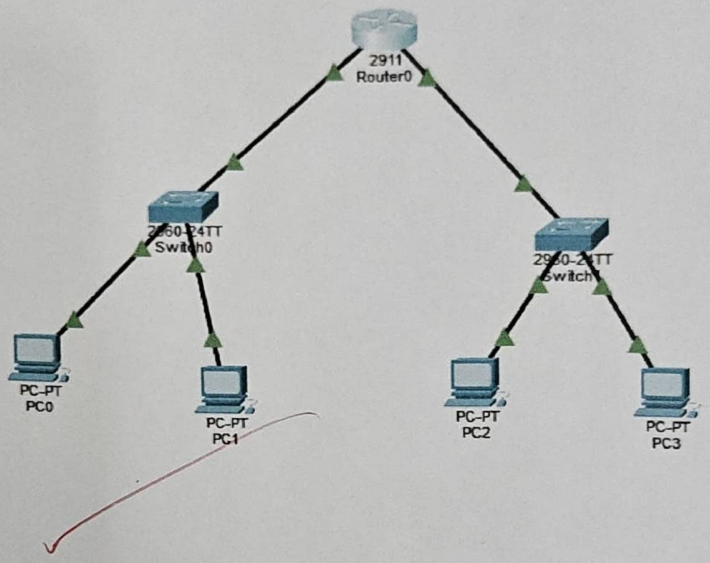
Configuring The Device:

```
#enable
#configure terminal
#interface FastEthernet 0/0
#ip address {IP address} {subnet mask}
#no shutdown
#exit
interface fastEthernet 0/1
ip address {IP addrs} {subnet mask}
no shutdown
exit
```

- The router interface is configured with the required IP address and subnet mask and interface is enabled
- The switch port connected to the PCs are configured in access mode
- Each PC is configured with an IP address, subnet mask, default gateway and DNS server. The PCs and router interface are assigned address from the same subnet
- Thus, the router, switch and PCs are configured succesfully for network communication

Testing the Network:

Now that our network topology is configured we can test the network open a command prompt on each PC and try to ping the other PC. If the ping issuccessful then the network is further properly. We can also use the ping command to test connectivity between the router and the PCs




```
Router>enable
Router#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.1.1     YES manual up                    up
GigabitEthernet0/1     192.168.2.1     YES manual up                    up
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
```

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.10

Pinging 192.168.1.10 with 32 bytes of data:

Reply from 192.168.1.10: bytes=32 time=3ms TTL=128
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128
Reply from 192.168.1.10: bytes=32 time=6ms TTL=128
Reply from 192.168.1.10: bytes=32 time=4ms TTL=128

Ping statistics for 192.168.1.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 6ms, Average = 3ms

C:\>
```

Understanding of subnetting

subnetting is the process of dividing a large IP network into smaller networks called subnet. It use a subnet mask divide the network into smaller section

Advantage of subnetting:

- Efficient use of IP addresses
- Reduces network traffic and congestion
- Improve network security
- Reduce broadcast traffic
- makes network management easier

subnetting in college:

Yes, subnetting is commonly implemented in college to seperate ~~such~~ network such as administrator, Faculty, computer Labs, Library, students and wifi

Result

Thus the implementation of subnetting in cisco Packet tracer simulator is executed succesfully and output is verified

Ex.No.: 10a Internetworking with Router in Cisco
Date: 14/5/26 packet tracer simulator

Aim

To internetworking with routers in CISCO Packet Tracer Simulator

Design and configure a simple internetwork using router

In this network, a router and 2PCs are used computer are connected with router using a copper straight through cable. After forming the network to check network connectivity a simple PDU is transfered from PC0 to PC1

Procedure:

step-1 (Configuring Router 1):

1. Select the router and Open CLI
2. Press ENTER to start configuring Router 1
3. Type enable to achive the privileged mode.

~~step-2~~ Router 1

Router > enable
Router # configure t
Enter config command one per line
Router (Config) # interface FastEthernet 0/0
Router (config-if) # ip addr 192.168.10.1.255.255.255.0
Router (config-if) # no shutdown
Router (config-if) # no shutdown

Router Config Table:

Device name	IP addr Fast ether	Subnet mask	IP addr	Subnet ma
Rotor1	192.168.10.1	255.255.255.0	192.168.20.1	255.255.255.0

PC configuration Table

Device Name	IP addr	Subnet mask	Gateway
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1

step 2 : (Configuring PC's):

1. Assign IP addrs to every PC in the Network
2. Select the PC, Go to the destop and select IP configuration and assign an IP addrs
3. Assign the defaut gateway of PC0 as 192.168.10.1
4. Assign the defaut gateway PC1 as 192.168.20.1

Step 3 : (Connect Pc with router):

1. Connect Fast ethernet 0 port of PC0 with Fast Ethernet 0/0 port Router 1
2. Connect Fast ethernet 0 port of PC1 with Fastethernet 0/1 port of Rotr 1

Result :

The two routers in cisco packet Tracer simulator is implemented sucessfully and Verfed Sucessfully

Ex.No.10.b Internetwork Using Wireless router DHCP
Date: 14/8/26 server and Internet Cloud.

Aim:

To design and configure an Internetwork using wireless router, DHCP server and Internet cloud in Cisco Packet Tracer and verify connectivity

Procedure:

Step 1: Build the Network

1. Place PC, Wireless Router, Laptop cable Modem Internet cloud and Cisco.com server
2. Rename the device as required
3. Connect
   - PC → Wireless Router Using copper straight-through cable
   - Wireless router → Cable Modem
   - Cable Modem → Internet Cloud
   - Internet Cloud → Cisco.com server

Step 2: Configure ~~Laptop~~ Wireless router

- ~~Replace~~ Set SSID: Home network
- Enable DHCP server
- Set DNS server: 208.67.220.220
- Save the settings

Step 3: Configure Laptop

* Replace Ethernet module with wireless WPC300N
* Power on the Laptop
* Go to Desktop → PC wireless
* Select Home network

Step 4: Config PC

Select DHCP

Step 5: Configure Internet Cloud

* Configure Coaxial → Ethernet
* Set the ethernet Provider network as Cable

Step 6: Configure Cisco.com server

DHCP:

Pool: DHCPpool

Gateway: 208.67.220.220

DNS: 208.67.220.220.

Start IP: 208.67.220.1

Subnet Mask: 255.255.255.0

Maximum User: 50

DNS

Name: Cisco.com

Type: A record

Addr: 208.67.220.220

Server IP: 208.67.220.220

Gateway: 208.67.220.1

DNS: 208.67.220.220

Step: 7 Verify connectivity using ping command

ping Cisco.com

Device	Interface	IP addr	Subnet Mask	Gateway
PC	Ethernet 0	DHCP (192.168.0.x)	255.255.255.0	192.168.0.1
Wireless Router	LAN	192.168.0.1	255.255.255.0	—
~~Wireless~~ ISP Wireless router	Internet	DHCP	—	—
Laptop	Wireless 0	DHCP	255.255.255.0	192.168.0.1
Cisco.com server	Ethernet0	208.67.220.220	255.255.255.0	208.67.220.1

Observation

1. Key features of wireless router and DHCP

* Wireless router: Provide WiFi, SSID, LAN

* DHCP: Automatically assigns IP address, subnet mask, gateway and DNS

2. Significance of DHCP

DHCP automatically provide network settings, reduce manual configuration and prevent IP conflicts

3. Internetwork Design

PC — wireless router — cable modem - Internet cloud - server

↓

Laptop

Result:

The internetwork was successfully designed and configured using a wireless router, DHCP server and internet cloud. The PC and laptop obtained IP address through DHCP and the ping to cisco.com was successfully verified.

expno:7 Develop a simple calculator using XMLRPC

Date:28/8/26

Aim:

To develop a simple calculator using XML-RPC in python, where the client send arithmetic operation request to server and recive the calculated result

Theory:

XML-RPC (XML Remote Procedure Call) is a protocol that allow a client to call functions located on a remote server. The request and response data are exchanged using XML.

Algorithm:

1. Start the XML-RPC server
2. Register arithmetic function on the server
3. Start the client and connect to the server
4. Display the calculator menu
5. Read the user's choice
6. If the choice is Exit, terminate the program
7. Otherwise, read two number
8. Send the operation request to the server
9. Recive and display the result
10. Repeat steps 4-9 until Exit is selected
11. Stop

Execute:

server.py — Server

```
from xmlrpc.server import SimpleXMLRPCServer
def add(a,b):
    return a+b
def subtract(a,b):
    return a-b
def multiply(a,b):
    return a*b
def divide(a,b):
    if b==0:
        return "Cannot divide by zero"
    return a/b
server = SimpleXMLRPCServer(("localhost", 8000))
print("Calculator server running on port 8000...")

server.register_function(add,"add")
server.register_function(subtract,"subtract")
server.register_function(multiply,"multiply")
server.register_function(divide,"divide")

server.serve_forever()
```

```
--- XML-RPC Calculator ---
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 1
Enter first number: 5
Enter second number: 3
======================
  Result: 8.0
======================

--- XML-RPC Calculator ---
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 2
Enter first number: 5
Enter second number: 2
======================
  Result: 3.0
======================

--- XML-RPC Calculator ---
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 3
Enter first number: 4
Enter second number: 3
======================
  Result: 12.0
======================
```

```
--- XML-RPC Calculator ---
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 4
Enter first number: 5
Enter second number: 3
========================
  Result: 1.6666666666666667
========================

--- XML-RPC Calculator ---
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 5
Exiting calculator...
```

client.py - client

```
import xmlrpc.client
proxy = xmlrpc.client.ServerProxy("http://localhost:8009/")
while True:
    print("\n--- XML-RPC calculator ---")
    print("1. Addition")
    print("2. Subtraction")
    print("3. Multiplication")
    print("4. Division")
    print("5. Exit")
    choice = input("Enter your choice: ")
    if choice == '5':
        print("Exiting calculator...")
        break
    if choice in ['1', '2', '3', '4']:
        a = float(input("Enter first number: "))
        b = float(input("Enter second number: "))
        if choice == '1':
            result = proxy.add(a, b)
        elif choice == '2':
            result = proxy.subtract(a, b)
        elif choice == '3':
            result = proxy.multiply(a, b)
        elif choice == '4':
            result = proxy.divide(a, b)
        print("-----------------")
        print("Result:", result)
        print("-----------------")
    else:
        print("Invalid choice! Please try again.")
```

Result:

Thus a simple calculator was sucessfuly implemented using XML-RPC in python and output is verified.

Exp.no: 8 Client-Server Architecture for communication

Date: 11/9/26

Aim

To implement a chat server and client architecture using TCP socket programming for communication between a client and a server

Steps/Procedure:

1. Import the socket module in both server and client program
2. Create a TCP socket using SOCK_STREAM
3. In server
   - * Bind the socket to IP address 127.0.0.1 and port 5000
   - * Listen for incoming client connection
   - * Accept the client connection
4. In the client
   - * Create a TCP socket
   - * connect to the server using IP address 127.0.0.1 and port 5000
5. The client send a message to the server
6. The server recieves and display the message
7. The server sends a replay to the client
8. The client recieves and display the replay
9. Continue the communication until either side send bye
10. Close the socket connection after the conversation ends

Code/Execution

server.py

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 5000))
server.listen(1)
print("server is waiting for client...")
conn, addr = server.accept()
print("client connected:", addr)
while True:
    message = conn.recv(1024).decode()
    if not message or message.lower() == "bye":
        print("Client disconnected")
        break
    print("Client:", message)
    replay = input("server: ")
    conn.send(replay.encode())
    if replay.lower() == "bye":
        break
conn.close()
server.close()
```

sender

[sender] connected to the server Type exit to quit

Enter msg to send: HI

[sender] message sent- waiting for receiver reply

[sender] Reply recied : Hello

server

[server] : server is running and waiting for connection

[server] sender connected from ('127.0.0.1', 63573)

[server] message from receiver : HELLO

[server] message from sender : HI

Receiver :-

[RECEIVER] connected to the server waiting for message

[RECEIVER] message : HI

Enter reply message: HELLOW

[RECEIVER] Reply sent waiting for next message

Client program:

```
import socket
import threading
host = "127.0.0.1"
port = 5000
client socket (socket.AF_INET, socket.SOCK_STREAM)
client.connect (HOST, PORT))
def receive_message():
    while True:
        try:
            msg = client.recv(1024).decode()
            if msg:
                print("\n" + msg)
                print("You: ", end="")
        except:
            break
thread.start()
print("Connected to chat server")
print("Type exit to leave the chat")
while True:
    msg = input("You: ")
    if msg.lower() == "exit":
        msg
        client.send(msg.encode())
Client.close
```

Result:

Thus the client-server chat system was successfully implemented using Socket programing. The output is verified.

Practical 11

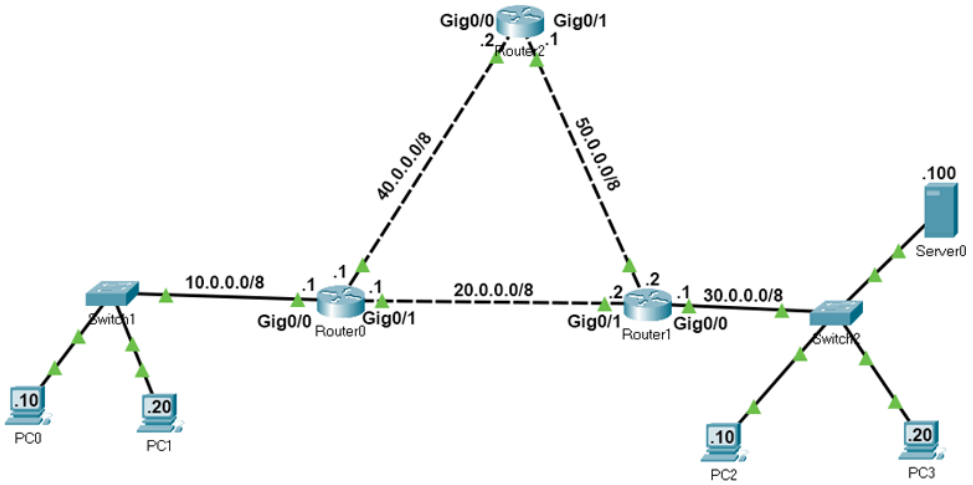
AIM:- a) Simulate Static Routing Configuration using CISCO Packet Tracer

Static routes are the routes you manually add to the router's routing table. The process of adding static routes to the routing table is known as static routing. Let's take a Packet Tracer example to understand how to use static routing to create and add a static route to the routing table.

Setting up a practice lab

Create a Packet Tracer lab as shown in the following image or download the pre-created lab and load it on Packet Tracer.

Packet Tracer Lab with Initial IP Configuration



In this lab, each network has two routes to reach. We will configure one route as the main route and another route as the backup route. If the link bandwidth of all routes is the same, we use the route that has the least number of routers as the main route. If the link bandwidth and the number of routers are the same, we can use any route as the main route and another route as the backup route.

If we specify two routes for the same destination, the router automatically selects the best route for the destination and adds the route to the routing table. If you manually want to select a route that the router should add to the routing table, you have to set the AD value of the route lower than other routes. For example, if you use the following commands to create two static routes for network 30.0.0.0/8, the router will place the first route in the routing table.

```
#ip route 30.0.0.0 255.0.0.0 20.0.0.2 10
#ip route 30.0.0.0 255.0.0.0 40.0.0.2 20
```

If the first route fails, the router automatically adds the second route to the routing table.

Creating, adding, verifying static routes

Routers automatically learn their connected networks. We only need to add routes for the networks that are not available on the router's interfaces. For example, networks 10.0.0.0/8, 20.0.0.0/8 and 40.0.0.0/8 are directly connected to Router0. Thus, we don't need to configure routes for these networks. Networks 30.0.0.0/8 and 50.0.0.0/8 are not available on Router0. We have to create and add routes only for these networks.

The following table lists the connected networks of each router.

Router	Available networks on local interfaces	Networks available on other routers' interfaces
Router0	10.0.0.0/8, 20.0.0.0/8, 40.0.0.0/8	30.0.0.0/8, 50.0.0.0/8
Router1	20.0.0.0/8, 30.0.0.0/8, 50.0.0.0/8	10.0.0.0/8, 40.0.0.0/8
Router2	40.0.0.0/8, 50.0.0.0/8	10.0.0.0/8, 20.0.0.0/8, 30.0.0.0/8

Let's create static routes on each router for networks that are not available on the router.

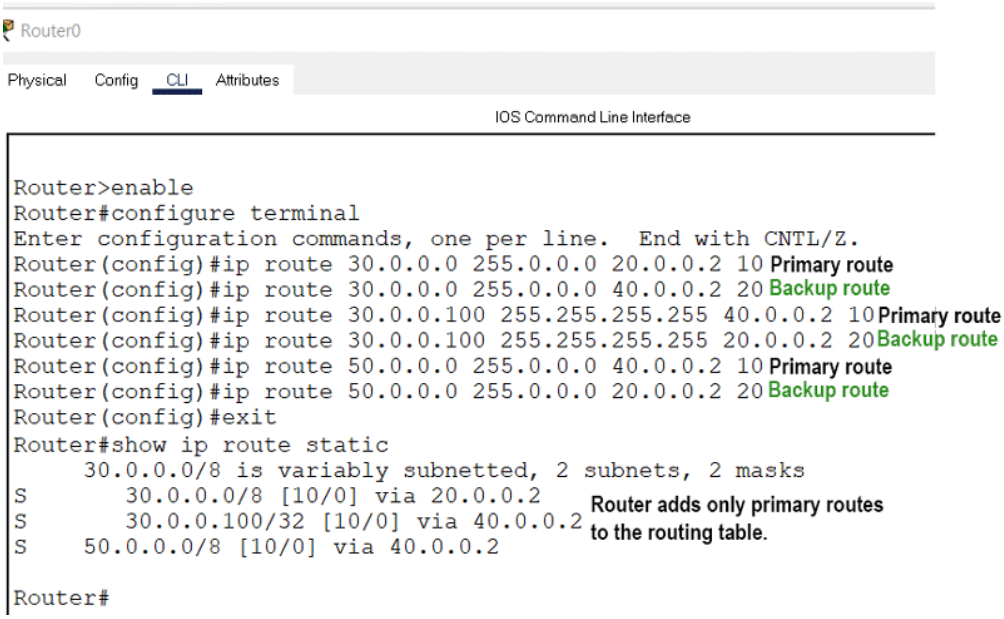
Router0 requirements

- Create two routes for network 30.0.0.0/8 and configure the first route (via Router1) as the main route and the second route (via Router2) as a backup route.
- Create two routes for the host 30.0.0.100/32 and configure the first route (via Router2) as the main route and the second route (via Router1) as a backup route.
- Create two routes for network 50.0.0.0/8 and configure the first route (via Router2) as the main route and the second route (via Router1) as a backup route.
- Verify the router adds only main routes to the routing table.

Router0 configuration

Access the CLI prompt of Router0 and run the following commands.

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 30.0.0.0 255.0.0.0 20.0.0.2 10
Router(config)#ip route 30.0.0.0 255.0.0.0 40.0.0.2 20
Router(config)#ip route 30.0.0.100 255.255.255.255 40.0.0.2 10
Router(config)#ip route 30.0.0.100 255.255.255.255 20.0.0.2 20
Router(config)#ip route 50.0.0.0 255.0.0.0 40.0.0.2 10
Router(config)#ip route 50.0.0.0 255.0.0.0 20.0.0.2 20
Router(config)#exit
Router#show ip route static
     30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S       30.0.0.0/8 [10/0] via 20.0.0.2
S       30.0.0.100/32 [10/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
Router#
```

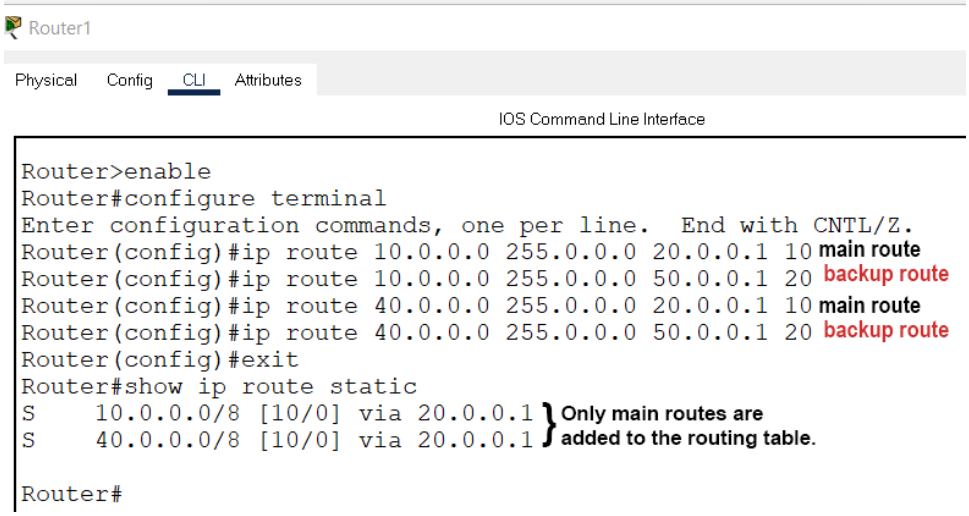


Router1 requirements

- Create two routes for network 10.0.0.0/8 and configure the first route (via Router0) as the main route and the second route (via Router2) as a backup route.
- Create two routes for network 40.0.0.0/8 and configure the first route (via Router0) as the main route and the second route (via Router2) as a backup route.
- Verify the router adds only main routes to the routing table.

Router1 configuration

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 10.0.0.0 255.0.0.0 20.0.0.1 10
Router(config)#ip route 10.0.0.0 255.0.0.0 50.0.0.1 20
Router(config)#ip route 40.0.0.0 255.0.0.0 20.0.0.1 10
Router(config)#ip route 40.0.0.0 255.0.0.0 50.0.0.1 20
Router(config)#exit
Router#show ip route static
S 10.0.0.0/8 [10/0] via 20.0.0.1
S 40.0.0.0/8 [10/0] via 20.0.0.1
Router#
```

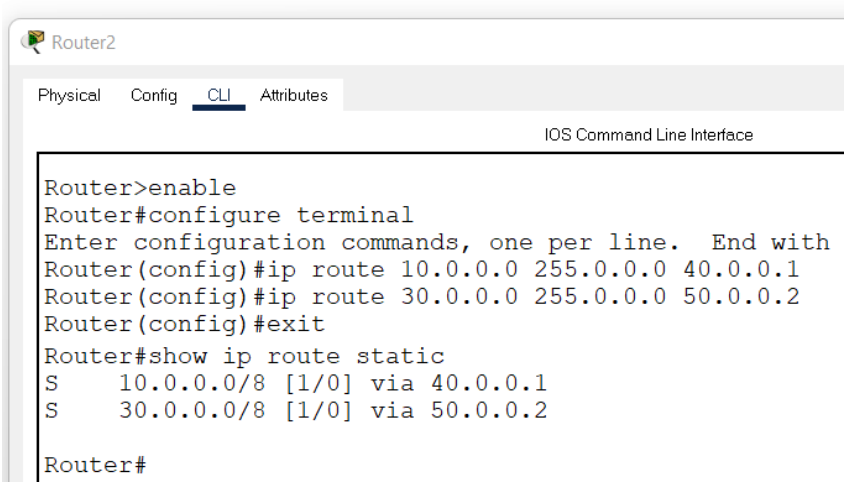


Router2 requirements

Create static routes for network 10.0.0.0/8 and network 30.0.0.0/8 and verify the router adds both routes to the routing table.

Router2 configuration

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 10.0.0.0 255.0.0.0 40.0.0.1
Router(config)#ip route 30.0.0.0 255.0.0.0 50.0.0.2
Router(config)#exit
Router#show ip route static
S 10.0.0.0/8 [1/0] via 40.0.0.1
S 30.0.0.0/8 [1/0] via 50.0.0.2
Router#
```



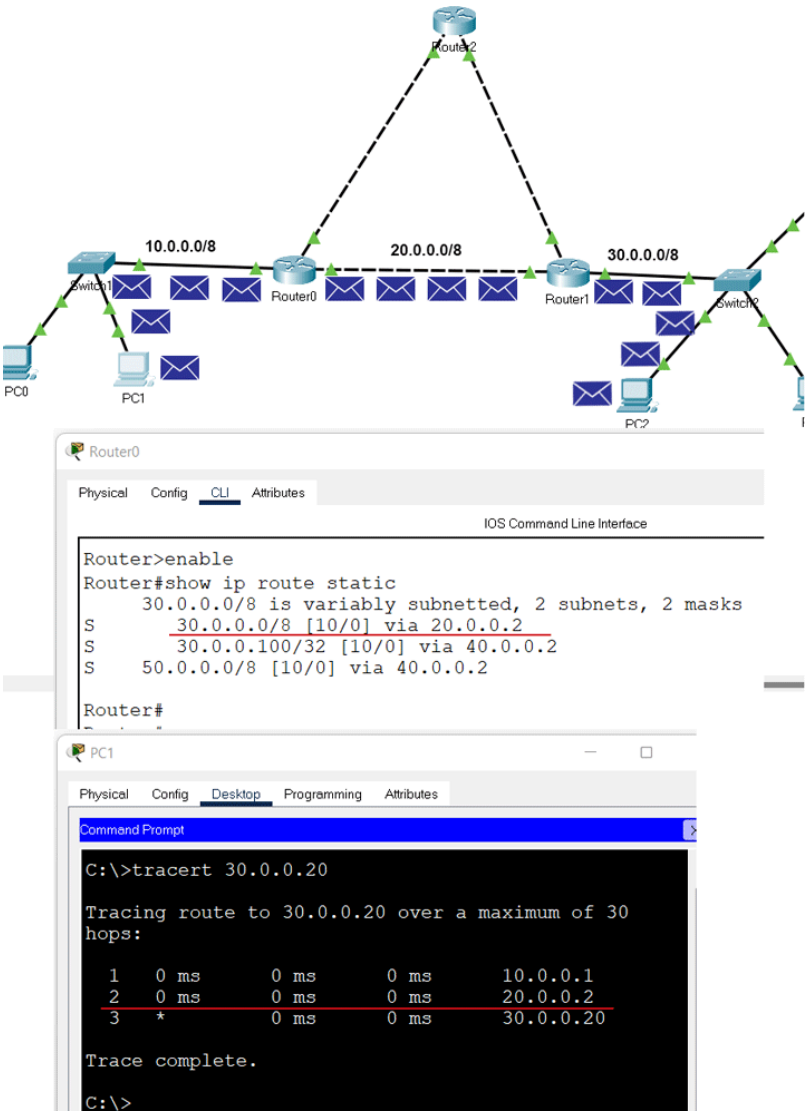
Verifying static routing

On Router0, we configured two routes for network 30.0.0.0/8. These routes are via Router1 and via Router2. We set the first route (via Router1) as the main route and the second route as the backup route. We can verify this configuration in two ways.

By sending ping requests to a PC of network 30.0.0.0/8 and tracing the path they take to reach it. For this, you can use the ‘tracert’ command on a PC of network 10.0.0.0/8. The ‘tracert’ command sends ping requests to the destination host and tracks the path they take to reach the destination.

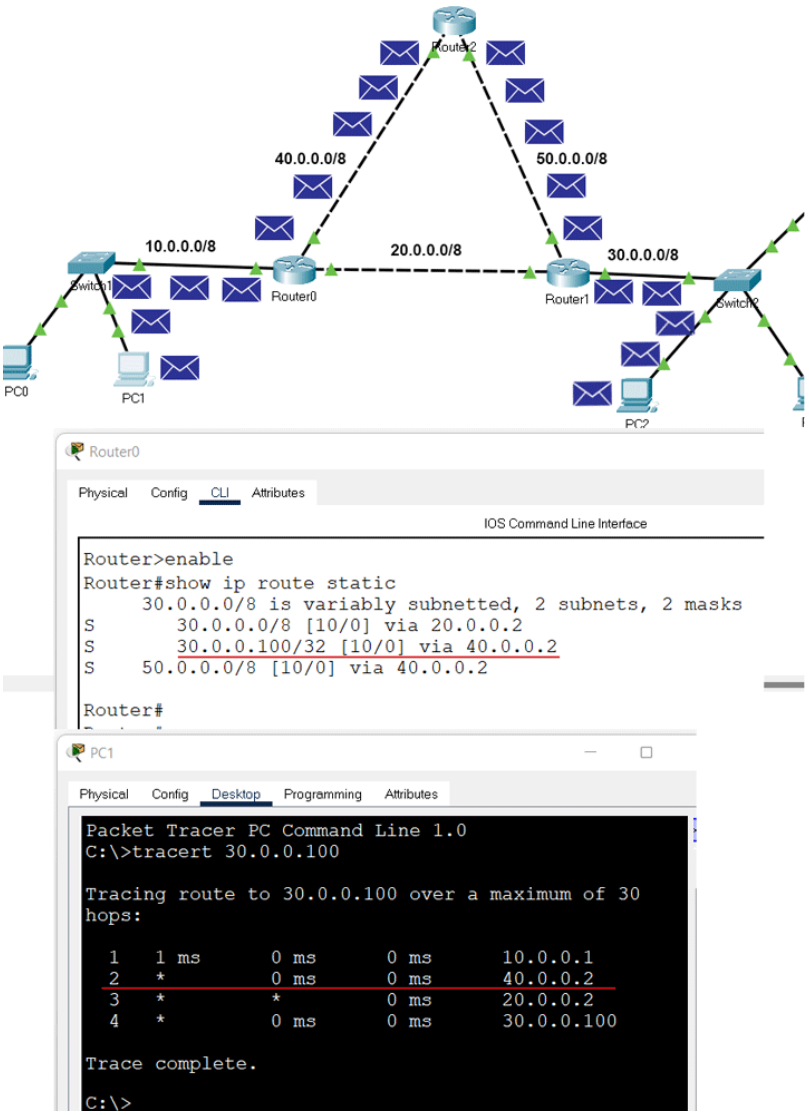
By listing the routing table entries on Router0. Since a router uses the routing table to forward data packets, you can check the routing table to figure out the route the router uses to forward data packets for each destination.

The following image shows the above testing.



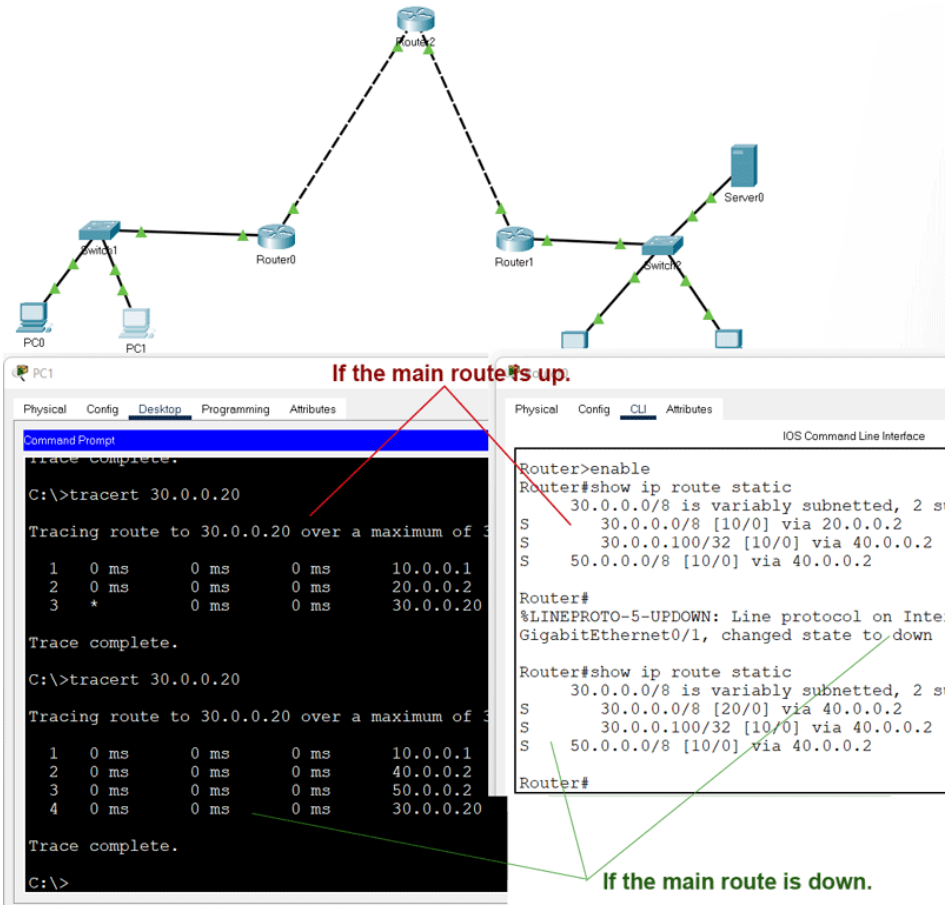
We also configured a separate static host route for the host 30.0.0.100/32. The router must use this route to forward data packets to the host 30.0.0.100. To verify this, you can do the same testing for the host 30.0.0.100.

The following image shows this testing.



We also configured a backup route for network 30.0.0.0/8. The router must put the backup route in the routing table and use it to forward data packets to network 30.0.0.0/8 when the main route fails. To verify this, we have to simulate the failure of the main route.

To simulate the failure of the main route, you can delete the link between Router0 and Router1. After deleting the link, do the same testing again for the network 30.0.0.0/8.



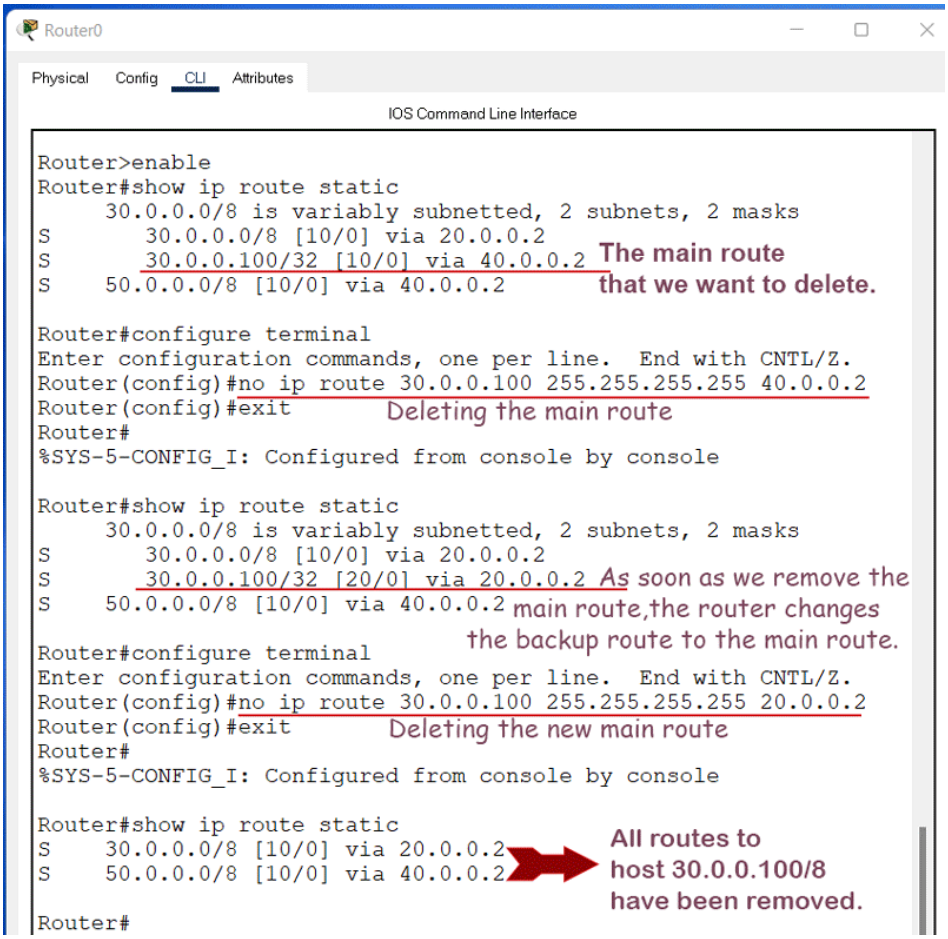
Deleting a static route

To delete a static route, use the following steps.

- Use the ‘show ip route static’ command to print all static routes.
- Note down the route you want to delete.
- Use the ‘no ip route’ command to delete the route.

If you have a backup route, the backup route becomes the main route when you delete the main route.

In our example, we have a backup route and a main route for the host 30.0.0.100/32. The following image shows how to delete both routes.


Router0
Physical
Config
CLI
Attributes
IOS Command Line Interface
Router>enable
Router#show ip route static
30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S 30.0.0.0/8 [10/0] via 20.0.0.2
S 30.0.0.100/32 [10/0] via 40.0.0.2
S 50.0.0.0/8 [10/0] via 40.0.0.2
The main route that we want to delete.
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#no ip route 30.0.0.100 255.255.255.255 40.0.0.2
Router(config)#exit
Deleting the main route
Router#
%SYS-5-CONFIG_I: Configured from console by console
Router#show ip route static
30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S 30.0.0.0/8 [10/0] via 20.0.0.2
S 30.0.0.100/32 [20/0] via 20.0.0.2
S 50.0.0.0/8 [10/0] via 40.0.0.2
As soon as we remove the main route,the router changes the backup route to the main route.
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#no ip route 30.0.0.100 255.255.255.255 20.0.0.2
Router(config)#exit
Deleting the new main route
Router#
%SYS-5-CONFIG_I: Configured from console by console
Router#show ip route static
S 30.0.0.0/8 [10/0] via 20.0.0.2
S 50.0.0.0/8 [10/0] via 40.0.0.2
All routes to host 30.0.0.100/8 have been removed.
Router#

Practical 11

AIM:- b) Simulate RIP using CISCO Packet Tracer

Initial IP configuration

Device	Interface	IP Configuration	Connected with
PC0	Fast Ethernet	10.0.0.2/8	Router0's Fa0/1
Router0	Fa0/1	10.0.0.1/8	PC0's Fast Ethernet
Router0	S0/0/1	192.168.1.254/30	Router2's S0/0/1
Router0	S0/0/0	192.168.1.249/30	Router1's S0/0/0
Router1	S0/0/0	192.168.1.250/30	Router0's S0/0/0
Router1	S0/0/1	192.168.1.246/30	Router2's S0/0/0
Router2	S0/0/0	192.168.1.245/30	Router1's S0/0/1
Router2	S0/0/1	192.168.1.253/30	Router0's S0/0/1
Router2	Fa0/1	20.0.0.1/30	PC1's Fast Ethernet
PC1	Fast Ethernet	20.0.0.2/30	Router2's Fa0/1

Assign IP address to PCs

Double click the PCs, click the Desktop menu item and click IP Configuration. Assign the IP address referring to the above table.

Assign IP address to interfaces of routers

Double click Router0, click CLI and press the Enter key to access the command prompt of Router0.

We need to configure an IP address and other parameters on interfaces before we can actually use them for routing. Interface mode is used to assign an IP address and other parameters. Interface mode can be accessed from global configuration mode. The following commands are used to access global configuration mode.

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#
```

From global configuration mode we can enter interface mode and configure the interface. The following commands assign an IP address on FastEthernet0/0.

```
Router(config)#interface fastEthernet 0/0
Router(config-if)#ip address 10.0.0.1 255.0.0.0
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#
```

- interface fastEthernet 0/0 command is used to enter interface mode.
- ip address 10.0.0.1 255.0.0.0 command will assign the IP address to the interface.
- no shutdown command will bring the interface up.
- exit command is used to return to global configuration mode.

A serial interface needs two additional parameters: clock rate and bandwidth. Every serial cable has two ends, DTE and DCE. These parameters are always configured at the DCE end. We can use the show controllers interface command from privileged mode to check the cable's end.

```
Router#show controllers serial 0/0/0
Interface Serial0/0/0
Hardware is PowerQUICC MPC860
DCE V.35, clock rate 2000000
[Output omitted]
```

The third line of output confirms that the DCE end of the serial cable is attached. If you see DTE here instead of DCE, skip these parameters. Now we have the necessary information, so let's assign IP addresses to the serial interfaces of Router0.

```
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface serial 0/0/0
Router(config-if)#ip address 192.168.1.249 255.255.255.252
Router(config-if)#clock rate 64000
Router(config-if)#bandwidth 64
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/1
Router(config-if)#ip address 192.168.1.254 255.255.255.252
Router(config-if)#clock rate 64000
Router(config-if)#bandwidth 64
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#
```

- Router#configure terminal – enters global configuration mode.
- Router(config)#interface serial 0/0/0 – enters interface mode.
- Router(config-if)#ip address 192.168.1.249 255.255.255.252 – assigns the IP address to the interface. For a serial link we usually use an IP address from a /30 subnet.
- clock rate 64000 and bandwidth 64 – in a real environment these parameters control the data flow between serial links and are set at the service provider's end. In a lab environment we can simply use these values.
- no shutdown – brings the interface up.
- exit – returns to global configuration mode.

We will use the same commands to assign IP addresses on the interfaces of the remaining routers. We need to provide clock rate and bandwidth only on the DCE side of a serial interface.

Router1

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface serial 0/0/0
Router(config-if)#ip address 192.168.1.250 255.255.255.252
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/1
Router(config-if)#ip address 192.168.1.246 255.255.255.252
Router(config-if)#clock rate 64000
```

```
Router(config-if)#bandwidth 64
Router(config-if)#no shutdown
Router(config-if)#exit
```

Use the same commands to assign IP addresses on the interfaces of Router2.

Router2

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface fastEthernet 0/0
Router(config-if)#ip address 20.0.0.1 255.0.0.0
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/0
Router(config-if)#ip address 192.168.1.245 255.255.255.252
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/1
Router(config-if)#ip address 192.168.1.253 255.255.255.252
Router(config-if)#no shutdown
Router(config-if)#exit
```

Now the routers have information about the networks that they have on their own interfaces. Routers will not exchange this information between themselves on their own. We need to implement the RIP routing protocol that will make them share this information.

Configure RIP routing protocol

Configuration of the RIP protocol requires only two steps.

- Enable the RIP routing protocol from global configuration mode.
- Tell the RIP routing protocol which networks you want to advertise.

Let's configure it on Router0.

Router0

```
Router0(config)#router rip
Router0(config-router)# network 10.0.0.0
Router0(config-router)# network 192.168.1.252
Router0(config-router)# network 192.168.1.248
```

The router rip command tells the router to enable the RIP routing protocol. The network command allows us to specify the networks which we want to advertise. We only need to specify the networks which are directly connected to the router. That is all we need to configure RIP. Follow the same steps on the remaining routers.

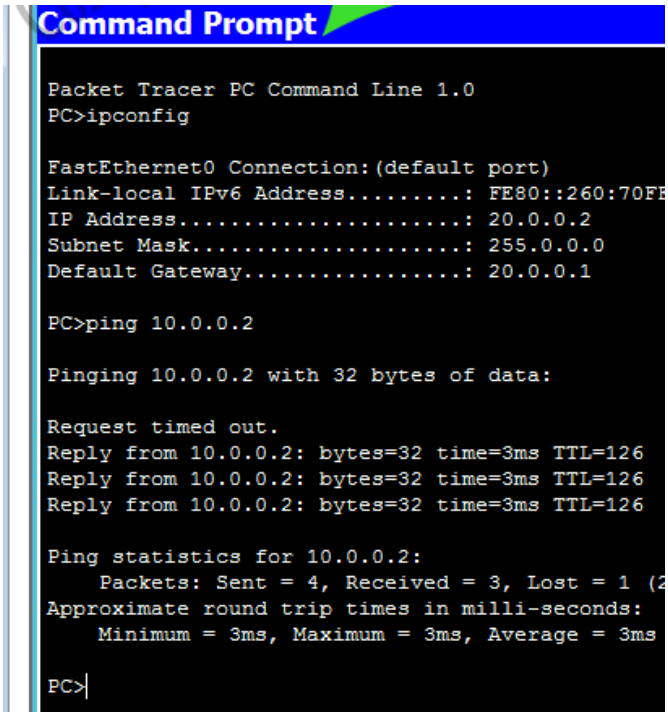
Router1

```
Router1(config)#router rip
Router1(config-router)# network 192.168.1.244
Router1(config-router)# network 192.168.1.248
```

Router2

```
Router2(config)#router rip
Router2(config-router)# network 20.0.0.0
Router2(config-router)# network 192.168.1.252
Router2(config-router)# network 192.168.1.244
```

That's it. Our network is ready to take advantage of RIP routing. To verify the setup we will use the ping command, which tests the connectivity between two devices. Access the command prompt of PC1 and use the ping command to test the connectivity to PC0.



RIP automatically manages all routes for us. If one route goes down, it automatically switches to another available route. To explain this process more clearly we have added one more route in our network. Currently there are two routes between PC0 and PC1.

Route 1

PC0 [Source / destination – 10.0.0.2] <==> Router0 [FastEthernet0/1 – 10.0.0.1] <==> Router0 [Serial0/0/1 – 192.168.1.254] <==> Router2 [Serial0/0/1 – 192.168.1.253] <==> Router2 [FastEthernet0/0 – 20.0.0.1] <==> PC1 [Destination / source – 20.0.0.2]

Route 2

PC0 [Source / destination – 10.0.0.2] <==> Router0 [FastEthernet0/1 – 10.0.0.1] <==> Router0 [Serial0/0/0 – 192.168.1.249] <==> Router1 [Serial0/0/0 – 192.168.1.250] <==> Router1 [Serial0/0/1 – 192.168.1.246] <==> Router2 [Serial0/0/0 – 192.168.1.245] <==> Router2 [FastEthernet0/0 – 20.0.0.1] <==> PC1 [Destination / source – 20.0.0.2]

By default RIP will use the route that has the lowest hop count between source and destination. In our network Route 1 has the lower hop count, so it will be selected. We can use the tracert command to verify it.

Now suppose Route 1 is down. We can simulate this situation by removing the cable attached between Router0 [S0/0/1] and Router2 [S0/0/1]. RIP will automatically reroute the traffic. Use the tracert command again to see dynamic routing in action.

Practical 12

AIM:- a) Implement echo client server using TCP/UDP sockets.

Algorithm:-

Server

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM).
2. Bind the socket to the server IP address and a port number (e.g., 127.0.0.1:8000).
3. Call listen() to wait for incoming client connections.
4. Call accept() to accept a connection from a client.
5. Receive the message from the client using recv().
6. Send the same message back to the client using send() (echo).
7. Repeat steps 5–6 until the client closes the connection or sends “exit”.
8. Close the client socket and the server socket.

Client

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM).
2. Connect to the server using the server IP address and port number.
3. Read a message from the user.
4. Send the message to the server using send().
5. Receive the echoed message from the server using recv() and display it.
6. Repeat steps 3–5 until the user types “exit”.
7. Close the socket.

Program (Python):-

echo_server.py

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("127.0.0.1", 8000))
s.listen(1)
print("Server is waiting for a connection...")
conn, addr = s.accept()
print("Connected by", addr)
while True:
    data = conn.recv(1024)
    if not data or data.decode().lower() == "exit":
        break
    print("Received:", data.decode())
    conn.send(data)          # echo back
conn.close()
s.close()
```

echo_client.py

```
import socket

c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(("127.0.0.1", 8000))
while True:
    msg = input("Enter message (exit to quit): ")
```

```
    c.send(msg.encode())
    if msg.lower() == "exit":
        break
    data = c.recv(1024)
    print("Echo from server:", data.decode())
c.close()
```

Input:-

Messages typed at the client terminal:

```
Hello Server
Computer Networks Lab
exit
```

Output:-

Server terminal

```
Server is waiting for a connection...
Connected by ('127.0.0.1', 52814)
Received: Hello Server
Received: Computer Networks Lab
```

Client terminal

```
Enter message (exit to quit): Hello Server
Echo from server: Hello Server
Enter message (exit to quit): Computer Networks Lab
Echo from server: Computer Networks Lab
Enter message (exit to quit): exit
```

Practical 12

AIM:- b) Implement chat client server using TCP/UDP sockets.

Algorithm:-

Server

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM).
2. Bind the socket to the server IP address and port number.
3. Call listen() and then accept() to establish a connection with the client.
4. Receive a message from the client using recv() and display it.
5. Read a reply from the server user and send it to the client using send().
6. Repeat steps 4–5 until either side sends “bye”.
7. Close the connection and the server socket.

Client

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM).
2. Connect to the server using the server IP address and port number.
3. Read a message from the client user and send it to the server.
4. Receive the server’s reply using recv() and display it.
5. Repeat steps 3–4 until either side sends “bye”.
6. Close the socket.

Program (Python):-

chat_server.py

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("127.0.0.1", 9000))
s.listen(1)
print("Chat server waiting for client...")
conn, addr = s.accept()
print("Client connected:", addr)
while True:
    msg = conn.recv(1024).decode()
    if not msg or msg.lower() == "bye":
        print("Client left the chat.")
        break
    print("Client:", msg)
    reply = input("Server: ")
    conn.send(reply.encode())
    if reply.lower() == "bye":
        break
conn.close()
s.close()
```

chat_client.py

```
import socket

c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(("127.0.0.1", 9000))
while True:
```

```
    msg = input("Client: ")
    c.send(msg.encode())
    if msg.lower() == "bye":
        break
    reply = c.recv(1024).decode()
    if not reply or reply.lower() == "bye":
        print("Server left the chat.")
        break
    print("Server:", reply)
c.close()
```

Input:-

Client

```
Hi
How are you?
bye
```

Server

```
Hello
I am fine
```

Output:-

Server terminal

```
Chat server waiting for client...
Client connected: ('127.0.0.1', 53102)
Client: Hi
Server: Hello
Client: How are you?
Server: I am fine
Client left the chat.
```

Client terminal

```
Client: Hi
Server: Hello
Client: How are you?
Server: I am fine
Client: bye
```

Practical 13

AIM:- Implement your own ping program

Algorithm:

1. Create a RAW socket using socket(AF_INET, SOCK_RAW, IPPROTO_ICMP) (requires administrator / root privileges).
2. Resolve the destination host name to an IP address.
3. Build an ICMP Echo Request packet: Type = 8, Code = 0, Identifier, Sequence Number and payload data.
4. Compute the Internet checksum of the packet and insert it into the ICMP header.
5. Record the send time and send the packet to the destination using sendto().
6. Wait for the reply using recvfrom() with a timeout.
7. Extract the ICMP header from the received packet (after the 20-byte IP header); if Type = 0 (Echo Reply) and the identifier matches, calculate the round-trip time (RTT) and TTL.
8. If no reply arrives before the timeout, display “Request timed out”.
9. Repeat for the required number of packets (e.g., 4) and display statistics: packets sent, received, lost, and minimum / maximum / average RTT.

Program (Python):-

```
import socket, struct, time, sys

def checksum(data):
    if len(data) % 2:
        data += b"\x00"
    s = sum(struct.unpack("!%dH" % (len(data) // 2), data))
    s = (s >> 16) + (s & 0xFFFF)
    s += s >> 16
    return ~s & 0xFFFF

def ping(host, count=4, timeout=2):
    ip = socket.gethostbyname(host)
    print(f"Pinging {host} [{ip}] with 32 bytes of data:")
    sock = socket.socket(socket.AF_INET, socket.SOCK_RAW,
socket.IPPROTO_ICMP)
    sock.settimeout(timeout)
    pid = 1234 & 0xFFFF
    rtts = []
    for seq in range(1, count + 1):
        header = struct.pack("!BBHHH", 8, 0, 0, pid, seq)
        payload = b"a" * 32
        chk = checksum(header + payload)
        header = struct.pack("!BBHHH", 8, 0, chk, pid, seq)
        start = time.time()
        sock.sendto(header + payload, (ip, 0))
        try:
            data, addr = sock.recvfrom(1024)
            rtt = (time.time() - start) * 1000
            ttl = data[8]
            icmp_type, _, _, rid, _ = struct.unpack("!BBHHH", data[20:28])
            if icmp_type == 0 and rid == pid:
                rtts.append(rtt)
```

```
                print(f"Reply from {addr[0]}: bytes=32 time={rtt:.0f}ms
TTL={ttl}")
        except socket.timeout:
            print("Request timed out.")
        time.sleep(1)
    sock.close()
    lost = count - len(rtts)
    print(f"\nPing statistics for {ip}:")
    print(f"    Packets: Sent = {count}, Received = {len(rtts)}, Lost =
{lost} ({lost*100//count}% loss)")
    if rtts:
        print(f"    Minimum = {min(rtts):.0f}ms, Maximum =
{max(rtts):.0f}ms, Average = {sum(rtts)/len(rtts):.0f}ms")

ping(sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1")
```

Input:-

```
sudo python3 myping.py 8.8.8.8
```

Output:-

```
Pinging 8.8.8.8 [8.8.8.8] with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=23ms TTL=117
Reply from 8.8.8.8: bytes=32 time=22ms TTL=117
Reply from 8.8.8.8: bytes=32 time=24ms TTL=117
Reply from 8.8.8.8: bytes=32 time=22ms TTL=117

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
    Minimum = 22ms, Maximum = 24ms, Average = 23ms
```

Practical 14

AIM:- Write a code using RAW sockets to implement packet sniffing.

Algorithm:

1. Create a RAW socket using socket(AF_PACKET, SOCK_RAW, ntohs(0x0003)) so that all Ethernet frames are captured (requires root privileges on Linux).
2. Enter a loop and receive packets using recvfrom(65535).
3. Unpack the first 14 bytes as the Ethernet header: destination MAC, source MAC and EtherType.
4. If EtherType = 0x0800 (IPv4), unpack the next 20 bytes as the IP header and extract version, header length, TTL, protocol, source IP and destination IP.
5. Based on the protocol field: 6 = TCP (extract source / destination ports, sequence number and flags), 17 = UDP (extract ports and length), 1 = ICMP (extract type and code).
6. Display the extracted details for each packet.
7. Continue until the user stops the program (Ctrl + C), then close the socket.

Program (Python):-

```
import socket, struct

def mac(b):
    return ":".join("%02x" % x for x in b)

sniffer = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
socket.ntohs(0x0003))
count = 0
try:
    while True:
        raw, _ = sniffer.recvfrom(65535)
        dst, src, proto = struct.unpack("!6s6sH", raw[:14])
        if proto != 0x0800:          # only IPv4
            continue
        count += 1
        ihl = (raw[14] & 0x0F) * 4
        ttl, p, _, s_ip, d_ip = struct.unpack("!8xBBH4s4s", raw[14:34])
        print(f"\nPacket {count}")
        print(f"Ethernet : {mac(src)} -> {mac(dst)}")
        print(f"IPv4     : {socket.inet_ntoa(s_ip)} ->
{socket.inet_ntoa(d_ip)}  TTL={ttl}")
        data = raw[14 + ihl:]
        if p == 6:
            sp, dp = struct.unpack("!HH", data[:4])
            print(f"Protocol : TCP   Src Port={sp}  Dst Port={dp}")
        elif p == 17:
            sp, dp = struct.unpack("!HH", data[:4])
            print(f"Protocol : UDP   Src Port={sp}  Dst Port={dp}")
        elif p == 1:
            t, c = struct.unpack("!BB", data[:2])
            print(f"Protocol : ICMP  Type={t}  Code={c}")
        else:
            print(f"Protocol : {p}")
except KeyboardInterrupt:
    print("\nSniffing stopped. Packets captured:", count)
    sniffer.close()
```

Input:-

Run the sniffer with root privileges and generate some traffic from another terminal (e.g., ping or opening a web page):

```
sudo python3 sniffer.py
ping 8.8.8.8
```

Output:-

```
Packet 1
Ethernet : 3c:52:82:aa:10:5f -> a4:5e:60:c1:22:7b
IPv4     : 192.168.1.10 -> 8.8.8.8  TTL=64
Protocol : ICMP  Type=8  Code=0

Packet 2
Ethernet : a4:5e:60:c1:22:7b -> 3c:52:82:aa:10:5f
IPv4     : 8.8.8.8 -> 192.168.1.10  TTL=117
Protocol : ICMP  Type=0  Code=0

Packet 3
Ethernet : 3c:52:82:aa:10:5f -> a4:5e:60:c1:22:7b
IPv4     : 192.168.1.10 -> 142.250.183.14  TTL=64
Protocol : TCP   Src Port=51522  Dst Port=443

^C
Sniffing stopped. Packets captured: 3
```

Practical 15

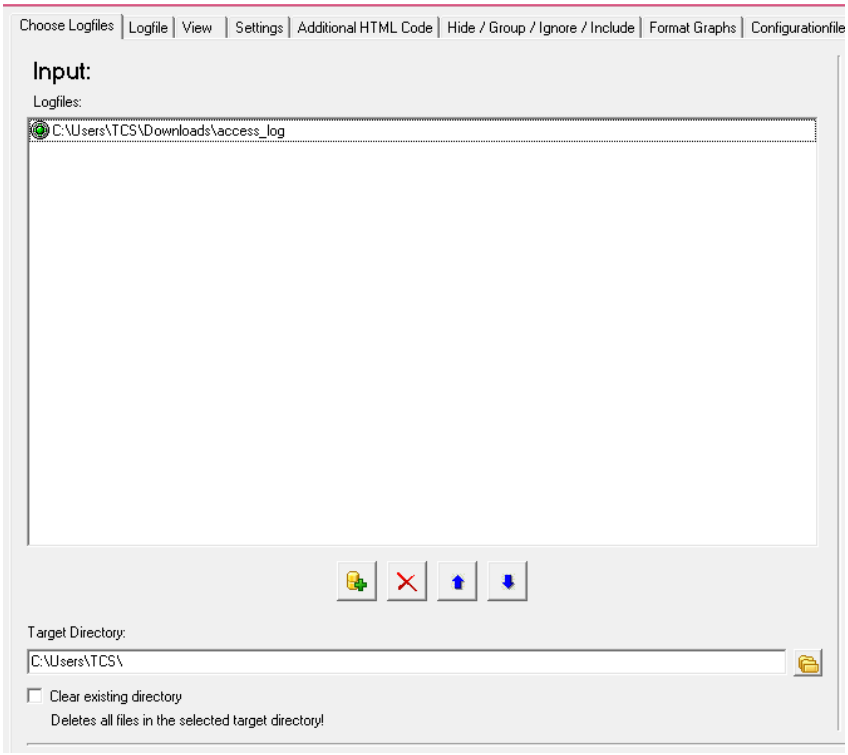
AIM:- To analyze the different types of web logs using Webalizer tool.

Procedure

Step 1: Run the Webalizer Windows version.

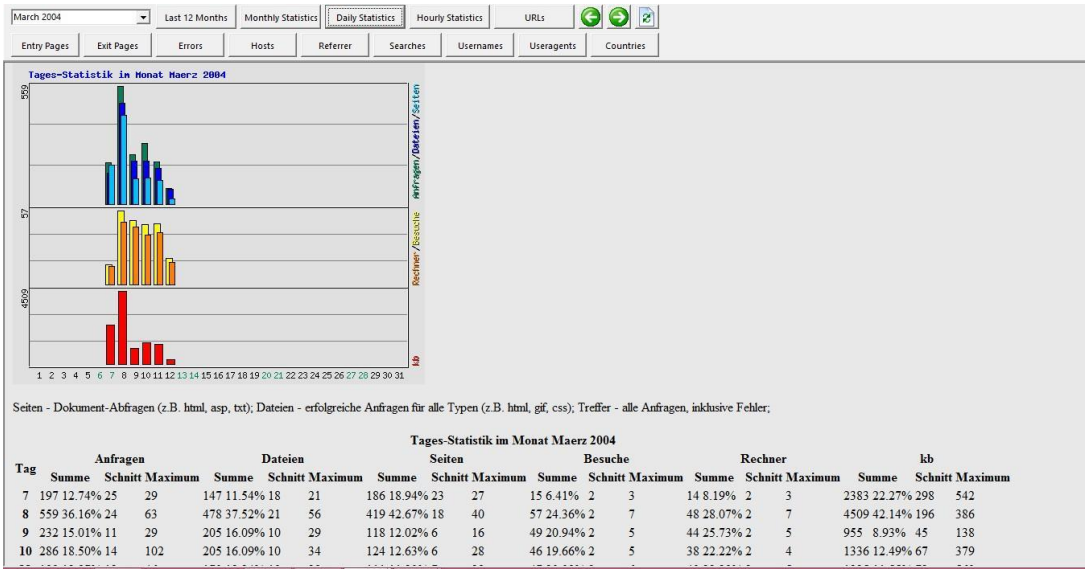
Step 2: Input the web log file (download from the web).

Step 3: Press Run Webalizer.



Output:

Monthly statistics

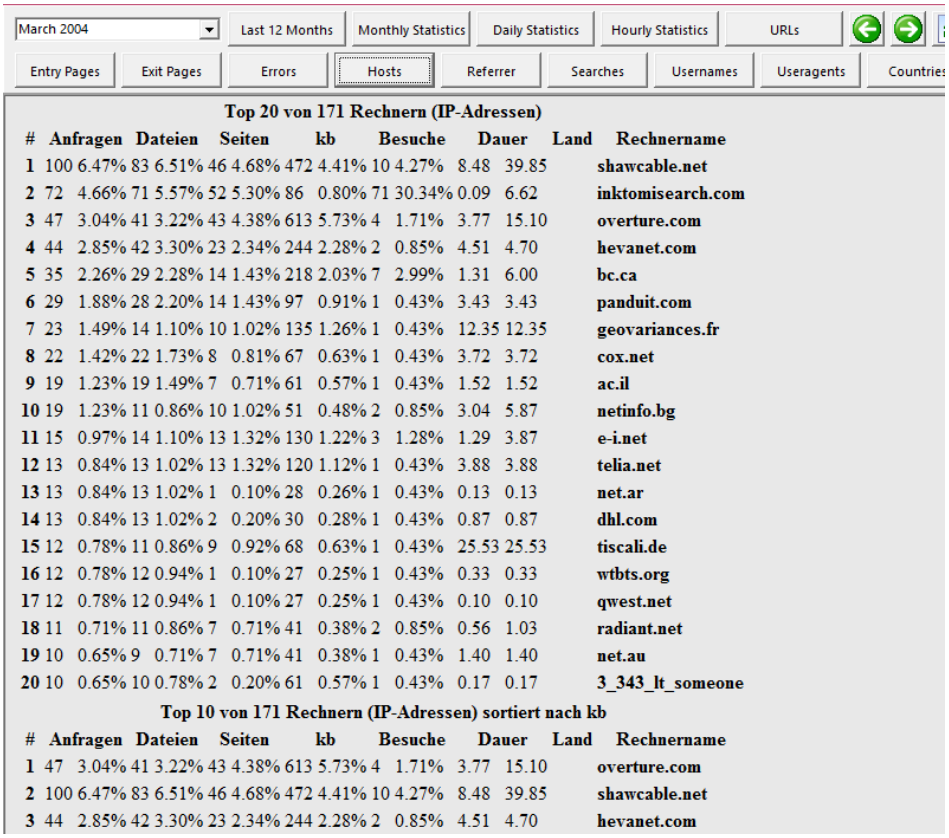


Seiten - Dokument-Abfragen (z.B. html, asp, txt); Dateien - erfolgreiche Anfragen für alle Typen (z.B. html, gif, css); Treffer - alle Anfragen, inklusive Fehler;

Tages-Statistik im Monat Maerz 2004

Tag	Anfragen			Dateien			Seiten			Besuche			Rechner			kb		
	Summe		Schnitt	Maximum	Summe		Schnitt	Maximum	Summe		Schnitt	Maximum	Summe		Schnitt	Maximum	Summe		Schnitt	Maximum
7	197	12.74%	25	29	147	11.54%	18	21	186	18.94%	23	27	15	6.41%	2	3	14	8.19%	2	3	2383	22.27%	298	542
8	559	36.16%	24	63	478	37.52%	21	56	419	42.67%	18	40	57	24.36%	2	7	48	28.07%	2	7	4509	42.14%	196	386
9	232	15.01%	11	29	205	16.09%	10	29	118	12.02%	6	16	49	20.94%	2	5	44	25.73%	2	5	955	8.93%	45	138
10	286	18.50%	14	102	205	16.09%	10	34	124	12.63%	6	28	46	19.66%	2	5	38	22.22%	2	4	1336	12.49%	67	379

Hosts



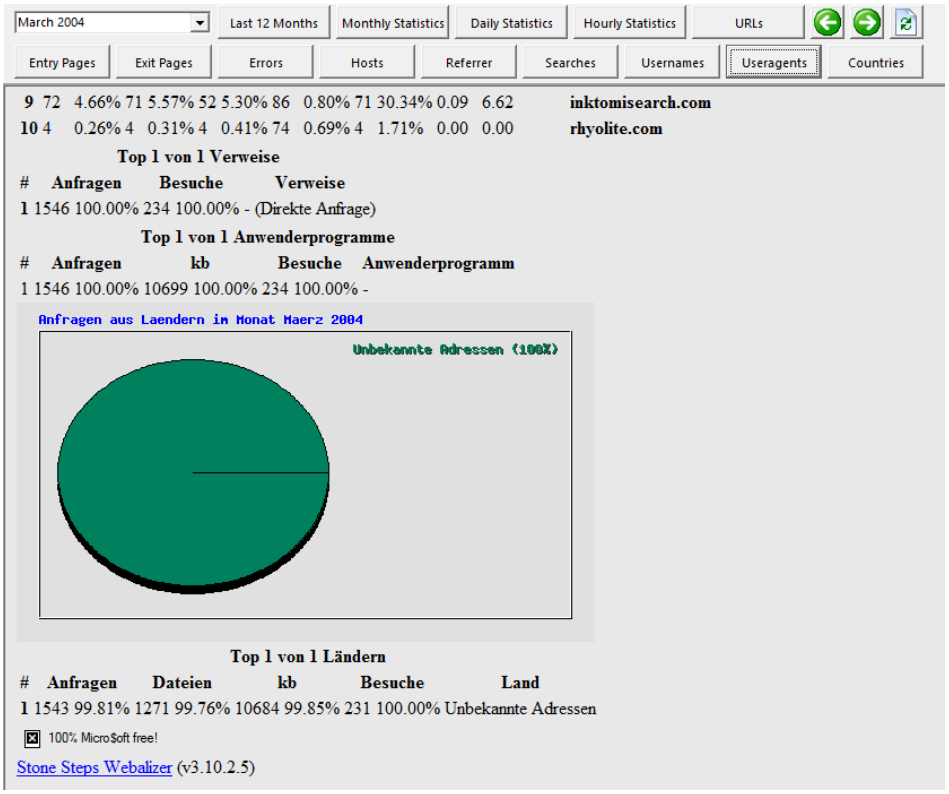
Top 20 von 171 Rechnern (IP-Adressen)

#	Anfragen		Dateien		Seiten		kb		Besuche		Dauer		Land	Rechnername
1	100	6.47%	83	6.51%	46	4.68%	472	4.41%	10	4.27%	8.48	39.85		shawcable.net
2	72	4.66%	71	5.57%	52	5.30%	86	0.80%	71	30.34%	0.09	6.62		inktomisearch.com
3	47	3.04%	41	3.22%	43	4.38%	613	5.73%	4	1.71%	3.77	15.10		overture.com
4	44	2.85%	42	3.30%	23	2.34%	244	2.28%	2	0.85%	4.51	4.70		hevanet.com
5	35	2.26%	29	2.28%	14	1.43%	218	2.03%	7	2.99%	1.31	6.00		bc.ca
6	29	1.88%	28	2.20%	14	1.43%	97	0.91%	1	0.43%	3.43	3.43		panduit.com
7	23	1.49%	14	1.10%	10	1.02%	135	1.26%	1	0.43%	12.35	12.35		geovariances.fr
8	22	1.42%	22	1.73%	8	0.81%	67	0.63%	1	0.43%	3.72	3.72		cox.net
9	19	1.23%	19	1.49%	7	0.71%	61	0.57%	1	0.43%	1.52	1.52		ac.il
10	19	1.23%	11	0.86%	10	1.02%	51	0.48%	2	0.85%	3.04	5.87		netinfo.bg
11	15	0.97%	14	1.10%	13	1.32%	130	1.22%	3	1.28%	1.29	3.87		e-i.net
12	13	0.84%	13	1.02%	13	1.32%	120	1.12%	1	0.43%	3.88	3.88		telia.net
13	13	0.84%	13	1.02%	1	0.10%	28	0.26%	1	0.43%	0.13	0.13		net.ar
14	13	0.84%	13	1.02%	2	0.20%	30	0.28%	1	0.43%	0.87	0.87		dhl.com
15	12	0.78%	11	0.86%	9	0.92%	68	0.63%	1	0.43%	25.53	25.53		tiscali.de
16	12	0.78%	12	0.94%	1	0.10%	27	0.25%	1	0.43%	0.33	0.33		wtbts.org
17	12	0.78%	12	0.94%	1	0.10%	27	0.25%	1	0.43%	0.10	0.10		qwest.net
18	11	0.71%	11	0.86%	7	0.71%	41	0.38%	2	0.85%	0.56	1.03		radiant.net
19	10	0.65%	9	0.71%	7	0.71%	41	0.38%	1	0.43%	1.40	1.40		net.au
20	10	0.65%	10	0.78%	2	0.20%	61	0.57%	1	0.43%	0.17	0.17		3_343_lt_someone

Top 10 von 171 Rechnern (IP-Adressen) sortiert nach kb

#	Anfragen		Dateien		Seiten		kb		Besuche		Dauer		Land	Rechnername
1	47	3.04%	41	3.22%	43	4.38%	613	5.73%	4	1.71%	3.77	15.10		overture.com
2	100	6.47%	83	6.51%	46	4.68%	472	4.41%	10	4.27%	8.48	39.85		shawcable.net
3	44	2.85%	42	3.30%	23	2.34%	244	2.28%	2	0.85%	4.51	4.70		hevanet.com

User-agents



9	72	4.66%	71	5.57%	52	5.30%	86	0.80%	71	30.34%	0.09	6.62		inktomisearch.com
10	4	0.26%	4	0.31%	4	0.41%	74	0.69%	4	1.71%	0.00	0.00		rhyolite.com

Top 1 von 1 Verweise

#	Anfragen		Besuche		Verweise
1	1546	100.00%	234	100.00%	- (Direkte Anfrage)

Top 1 von 1 Anwenderprogramme

#	Anfragen		kb		Besuche		Anwenderprogramm
1	1546	100.00%	10699	100.00%	234	100.00%	-

Top 1 von 1 Ländern

#	Anfragen		Dateien		kb		Besuche		Land
1	1543	99.81%	1271	99.76%	10684	99.85%	231	100.00%	Unbekannte Adressen

100% MicroSoft free!

Stone Steps Webalizer (v3.10.2.5)